



University of Pittsburgh

CS 1632 Introduction

Instructor Introduction

Course Introduction

Wonsun Ahn

Department of Computer Science

School of Computing and Information





Instructor Introduction



My Technical Background

■ Wonsun Ahn

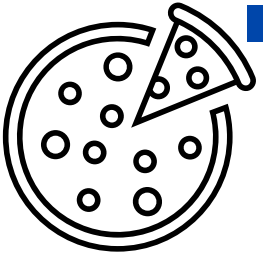
- First name is pronounced *one-sun* (if you can manage)
- Or you can just call me Dr. Ahn (rhymes with *naan*)

■ Relevant Experience

- Bluebird Corporation (70-person startup company)
 - Manufactures industrial hand-held devices from top to bottom
 - Me: Built software stack based on Windows Embedded
- IBM Research (thousands of people)
 - Does next-gen stuff like carbon nanotubes, quantum computers
 - Me: Built software simulators that model supercomputer designs

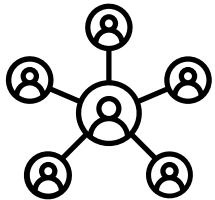


Zero-sum Thinking No Longer Works



■ Zero-sum thinking (old way of thinking)

- “If you get a larger slice of the pie, I get a smaller slice.”
- Therefore, if you lose, I win (and vice versa)



■ Increasingly, everything is connected

- Pandemic: If my neighbors catch the virus, so will I
- Environment: If disaster hits my neighbor, I feel the effects
- Economy: Think of how the subprime mortgage crisis spread



■ Zero-sum thinking no longer works

- If you lose, I lose as well



Collaboration Beats Competition

- Truer in all spheres as the world gets more connected
- Collaboration is also happening in the IT industry
 - The *open-source* movement
 - Increasing importance of the software/hardware *ecosystem*
 - Increasing importance of the developer *community*
- Collaboration is also important for learning
 - During my undergrad years, what do I remember best?
 - Stuff that my classmates taught me
 - Stuff that I explained to my classmates



Supporting Collaborative Learning

- You will be working with a partner (on GitHub)
 - You will learn how to collaborate on a source repository

- You are a member of the CS 1632 Team
 - I encourage you to be on Teams at most times (I will too)
 - Recommend you install app on both laptop and cell phone
 - You can ask questions on the appropriate Teams channel
 - Either your classmate or your instructor will answer
 - You can also chat with any individual on the Team
 - “Manage Team” item in the “...” Team context menu



Supporting Collaborative Learning

- What to share and what not to share
 - You can freely discuss Exercises
 - Posting your code relevant to the discussion is totally fine
 - You ***cannot*** share code for the Final Project
 - Questions limited to understanding the parameters of the project
 - Once you fully understand the exercises, no need!

- Activity on Teams results in extra participation points
 - Asking questions and answering both count as activity!
 - Doesn't have to be questions. Random comments, observations, stuff you read online are welcome too.



Course Introduction



Structure of the Course

■ Formative Assessments

- 30%: 10 exercises
 - You are expected to learn and get hands-on experience
- 30%: 7 quizzes (one with lowest score dropped)
 - You are expected to demonstrate what you learned from the exercises

■ Summative Assessments

- 20%: Final Project
- 10%: Final Exam

■ Attendance

- 5%: TopHat attendance
- 5%: In-class / Teams participation



For More Details

- Please refer to the course repository:
https://github.com/wonsunahn/CS1632_Fall2026
 - Link also on the Canvas syllabus menu

- Class resources:
 - Canvas: announcements, assignments
 - GitHub: syllabus, textbook PDF, lectures, exercises, deliverables
 - Tophat: attendance
 - GradeScope: exercise / project submission, grading and feedback
 - Microsoft Teams: Out-of-class communication



What is Software Quality Assurance (QA)?

All activities that ensure quality during software development

What QA activities have you done before?



What QA techniques do you want to learn?





QA includes....

Unit testing, systems testing, user acceptance testing, requirements validation, KPIs, combinatorial testing, stochastic testing, performance testing, software profiling, penetration testing, static analysis, linting, security analysis, software verification, model checking, usability analysis, defect tracking, TDD (Test-Driven Development), DevOps (Development and Operations), SRE (Site Reliability Engineering), AI-assisted QA, QA for AI software, ...

It touches most of computer science!



Case Study: Boeing 737 MAX Crashes

15

Lion Air crash: Boeing 737 plane crashes in sea off Jakarta

🕒 29 October 2018

[f](#) [💬](#) [🐦](#) [✉](#) [Share](#)

ALEX DAVIES TRANSPORTATION 03.10.2019 02:47 PM

Crashed Ethiopian Air Jet Is Same Model as Lion Air Accident

An Ethiopian Airlines Boeing 737 MAX 8 crashed shortly after takeoff Sunday, evoking comparisons to an Indonesian incident in October.

[Boeing & Aerospace](#) | [Business](#) | [Nation & World](#) | [Times Watchdog](#)

Flawed analysis, failed oversight: How Boeing, FAA certified the suspect 737 MAX flight control system

March 17, 2019 at 6:00 am | Updated March 21, 2019 at 9:46 am

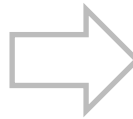
[f](#) [✉](#) [🐦](#)



Case Study: Boeing 737 MAX Crashes

16

- How was Boeing 737 MAX different from previous 737 models?
- First Boeing 737 (737-100) was built in April, 1967
 - Had a low profile to ease loading/unloading of plane
(They didn't have belt-loading baggage vehicles at that time)
- Boeing 737 MAX was built in December, 2018
 - Reused old design with larger engine for heavier load (to cut costs)
 - Engine did not fit under wing so had to bring it upwards and forwards





Case Study: Boeing 737 MAX Crashes

17

- New engine placement on 737 MAX led to worse aerodynamics
- Boeing did not want to retrain pilots (to cut costs)
- Boeing chose instead to write software to emulate an old 737
 - Make it “feel” like pilot was flying an old 737 instead of 737 MAX
 - Called *Maneuvering Characteristics Augmentation System* (MCAS)
- MCAS was the culprit that forced the planes into a nosedive
- MCAS had software quality issues at multiple levels
 - Requirements validation (used loopholes to skirt FAA oversight)
 - Robustness testing (skirted single-point-of-failure testing)
 - Defect reporting (catastrophic failures reported as merely hazardous)
- The root problem was that of corporate culture
 - Increased competition from Airbus since 2000 led to focus on cutting costs
 - First Boeing product to outsource critical flight software to 3rd parties